Useful warning before a service fails

ContinuityGuard - real-time system anomaly detection and evidence-backed operator support on IBM LinuxONE

DETECT

Forecast a developing service-objective breach from causal rolling telemetry.

UNDERSTAND

Inspect the evidence and the dependent services that matter.

PREVIEW A RESPONSE

Compare an operator action with no action in the same controlled workload.

37 s median early warning in the final synthetic test

Hospital digital-service availability

A shared infrastructure bottleneck can affect several essential software services before the operator has a useful explanation.

The user: hospital IT operators with limited monitoring capacity.

The decision: which dependency to investigate, which services need protection, and which runbook to review.

Real-world context: NHS England documented postponed care after the 2024 Synnovis incident. This prototype was not tested on that incident.

A native LinuxONE decision path

Training, evaluation, inference, explanation and the API run on the provided s390x VM. The browser connects through an SSH tunnel.

Telemetry
Validated queue

>

Causal features
Past/current only

>

Forecast + novelty
Separate scores

>

Incident evidence
Runbook + impact

Predictive AI

Calibrated seven-leaf boosted-tree forecast; Isolation Forest for novelty; thresholds as a measured baseline and failure fallback.

Predictive + generative AI

Local Granite selects evidence-grounded wording under a strict schema. No tool access or autonomous remediation.

RHEL 9.8 / 3 virtual CPUs / 7.3 GiB RAM / Podman / Jupyter / scikit-learn

Learn, calibrate, select, then test

The same causal feature code is used in Jupyter and online inference. Entire runs stay in one split.

60
Training runs

18
Calibration runs

24
Validation runs

60
Final test runs

26 inputs describe current signals, rolling means/slopes, load-normalized values and service identity. Fault labels, seeds, onset time and future values never enter the predictors.

Three forecast candidates and alert policies were compared on validation. The earlier inspected development report is retained. Final v2 testing uses fresh seeds; simulator families still overlap.

Earlier warning, with a precision tradeoff

Final test: 60 synthetic runs, 47 service incidents, 10 healthy service-hours. Service incidents in one run are correlated.

Detector	Early recall	Precision	Median lead*	Missed
Static thresholds	89.4%	68.3%	9 s	4 / 47
Robust statistics	63.8%	20.3%	36.5 s	17 / 47
Isolation Forest	85.1%	26.7%	46 s	7 / 47
ContinuityGuard	**91.5%**	**61.4%**	**37 s**	**4 / 47**

*Among early detections. Both final model and threshold baseline had zero alerts in tested healthy runs; the model had 27 unmatched alerts across all runs, versus 20 for thresholds.

Unseen abrupt stress: 24 / 24 incidents detected, zero warned in advance. No precursor means no promised early warning.

A decision an operator can inspect

The contribution connects warning evidence, declared software-service importance, and a repeatable response preview.

01 / Evidence

Measured pool, queue, retry and latency values accompany the alert. The dependency diagnosis is a hypothesis.

02 / Response preview

The same future demand/noise is replayed with and without an allowlisted simulator action.

03 / Explain safely

Local generation uses valid evidence IDs and runbook wording; validation failure is explicit.

Example held-out paired experiment: seed 50300

+294 modeled urgent-results successes / 100 s to 0 s breach duration / 1,418 batch requests deferred

Simulator-conditional effects, not proof of a real hospital intervention. No host or real service is modified.

Reproducible evidence and clear limits

A prototype designed to be reviewed, rerun and challenged.

Runtime evidence

At 100 synthetic window events/s: 9.03–53.60 ms p95 across two runs processing per four-event engine batch; about 186 MB process RSS. Short eight-second test, with generation off.

This excludes HTTP, queue waiting and browser rendering. It does not establish maximum capacity or production resilience.

Review package

Executed Jupyter notebooks, trained model hash, split manifest, tests, measured reports and readable source.

Next: authorized real telemetry, operator feedback, stronger authentication and broader failure testing. No z/OS integration, hardware accelerator use or clinical benefit is claimed.

Repository: github.com/ANBU-SELVAN-1907/RASCALS-IBM-DATATHON

AI-assisted implementation; team review and organizer guidance apply. Open-source resources are attributed in the repository.